

發球機

Madina 發明了一台發球機，用來娛樂 IOI 的參賽者。該機器的內部結構如下：

- 機器由 N 個節點組成，索引範圍從 0 到 $N - 1$ 。節點 $N - 1$ 稱為 **根節點**。
- 對於每個節點 u （其中 $0 \leq u < N - 1$ ），節點 u 的 **父節點** 為節點 $P[u]$ （其 $P[u] > u$ ），而節點 u 則是 $P[u]$ 的 **子節點**。根節點沒有父節點。沒有子節點的節點稱為 **葉節點**。

你不知道 N ，也不知道任何節點 u 的父節點 $P[u]$ 。相反地，Madina 告訴你葉節點的數量為 M ，以及葉節點的索引為 $0, 1, \dots, M - 1$ 。你的任務是透過操作這台機器來推斷其內部結構。

機器中的每個節點最多可容納一個 **球**，每個球都有一個 **值**，該值為非負整數。若某節點所容納的球之值為 x ，則稱該節點的值為 x 。若節點包含一個球，則該節點為 **佔用** 狀態；否則，即為 **空閒** 狀態。初始時，所有節點皆為空閒。

您可以執行兩類操作：

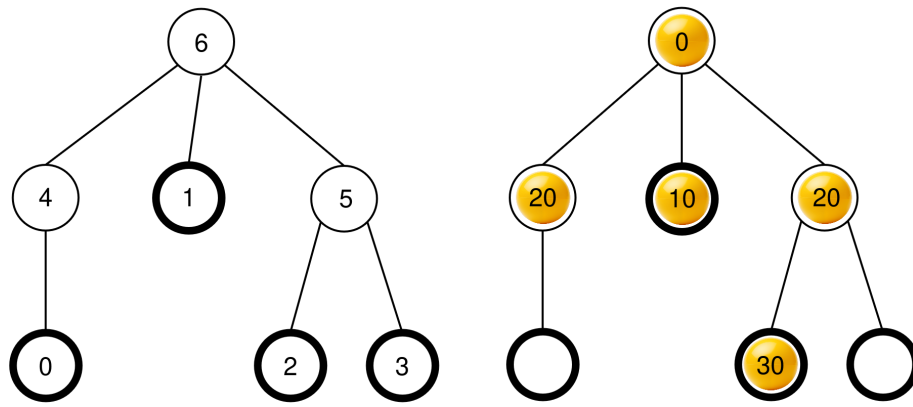
- **insert(U, X)**：嘗試將值 X 的新球體插入到葉節點 U 。
 - 若葉節點 U 已被佔用，則操作會返回 `false` 而不插入該球體。
 - 若葉節點 U 為空閒，則將球體放置於節點 U 。接著，只要當前節點的父節點為空，球體便會反覆移動至該父節點。當球體抵達根節點，或抵達其父節點已被佔用的節點時，移動即停止。此操作會傳回 `true`。
- **collect()**：若根節點為空（表示機器為空），則 `collect` 會傳回一個空陣列。否則，下文所述的遞歸程序 `traverse` 會將目前位於機器中的所有球的值收集到陣列 S 中。初始時，陣列 S 為空，並會呼叫 `traverse(N - 1)`。

```
traverse(u):  
    將節點 u 處的球體值追加至 S 的末尾  
    設 c[u] 是 u 的已佔用子節點清單  
    根據這些節點處球體的值，將 c[u] 按非遞減順序排序  
        （若多個節點具有相同數值，則可按任意順序排列）  
    對於 c[u] 中的每個 v:  
        traverse(v)
```

此程序完成後，**所有球都會從機器中移除**，且 `collect` 會傳回 S 。

舉例來說，假設有一台擁有 $N = 7$ 個節點的機器，其父節點陣列為 $P = [4, 6, 5, 5, 6, 6]$ 。左側圖片顯示了帶有節點索引的空機器，而右側圖片則展示了一系列 `insert` 操作後球體的可能配置（此操作序列詳見

「範例」章節)。



當呼叫`collect()`時，根節點（節點6）已被佔用，因此 S 被初始化為 $S = []$ ，並呼叫`traverse(6)`。

traverse(6)：節點6的值為0；將其附加至 S 後，結果為 $S = [0]$ 。節點6的子節點分別為4、1及5，這些節點均已被佔用。它們的值分別為20、10及20。按非遞減順序排序後，結果為 $c[6] = [1, 5, 4]$ （請注意， $c[6] = [1, 4, 5]$ 也是有效的，因為節點4和5的值相同）。接著，此程序會依此順序對 $c[6]$ 中的每個節點呼叫`traverse`。

- **traverse(1)**：節點1的值為10；將其附加至 S 後，結果為 $S = [0, 10]$ 。節點1 沒有被佔用的子節點，因此此呼叫結束。
- **traverse(5)**：節點5的值為20；將其附加至 S 後，得到 $S = [0, 10, 20]$ 。節點5 有一個被佔用的子節點，即節點2，因此 $c[5] = [2]$ 並呼叫程序`traverse(2)`。
 - **traverse(2)**：節點2的值為30；將其附加至 S 後，得到 $S = [0, 10, 20, 30]$ 。節點2沒有被佔用的子節點，因此此呼叫結束。
 - 執行流程返回至`traverse(5)`，該節點已處理完 $c[5]$ 中的所有子節點，因此此呼叫結束。
- **traverse(4)**：節點4的值為20；將其追加至 S 後，結果為 $S = [0, 10, 20, 30, 20]$ 。節點4沒有被佔用的子節點，因此此呼叫結束。

所有呼叫均已完成，且沒有更多節點需要處理。最後，所有球體皆從機器中移除，且`collect` 會傳回陣列 $S = [0, 10, 20, 30, 20]$ 。

您的任務是確定節點數量 N ，並回報一個描述機器結構的父節點陣列 $R = [R[0], R[1], \dots, R[N-2]]$ ，但對於非葉節點且非根節點的節點，其標籤可能有所不同。正式來說，如果能夠為機器的每個節點 u 分配一個不同的標籤 $L[u]$ ，且 $0 \leq L[u] < N$ ，則稱所報告的數組 R 是正確的：

- 對於所有 $0 \leq u < M$ 和 $u = N-1$ ，皆滿足 $L[u] = u$ ；以及
- 對於所有 $0 \leq u < N-1$ ，皆滿足 $L[P[u]] = R[L[u]]$ 。

無需滿足 $R[u] > u$ 。回報解法時，機器 必須為空。

設 K 為執行的`collect`操作次數， B 為所有`insert`呼叫中任意球的最大值。設 $C = K + B$ 。 C 不得超過1000。您在某些子任務中的得分取決於 C 的值。

實作細節

您應實作以下程序。

```
std::vector<int> find_structure(int M)
```

- M ：機器中的葉節點數量。
- 此程序針對每個測試案例會被呼叫 **exactly once**。
- 此程序必須回傳一個長度為 $N - 1$ 的陣列 $R = [R[0], R[1], \dots, R[N - 2]]$ ，此陣列應為正確的父節點陣列。

為了與機器互動，您的程序可以呼叫以下兩個程序。

第一個程序是：

```
bool insert(int U, int X)
```

- U ：應將球插入的葉節點索引。必須滿足條件 $0 \leq U < M$ 。
- X ：球的整數值。必須滿足條件 $0 \leq X \leq 1000$ 。
- 若球成功插入，此程序會傳回 `true`；若葉節點 U 已佔用，則傳回 `false`。
- 此程序最多可被呼叫 **500 000** 次。

第二個程序如下：

```
std::vector<int> collect()
```

- 收集所有已插入球的數值，清空機器，並傳回已收集的陣列 S 。

若在程式執行期間的任何時刻， C 的值超過 **1000**，您的解法將被判為 `Output isn't correct: Too many resources used`。

在呼叫 `find_structure` 之前，機器的結構就已經確定了。評分程式是**確定性的**，也就是說，如果運行兩次，並且兩次運行都執行相同的操作序列，那麼對 `collect` 的呼叫將傳回相同的陣列。

限制條件

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

子任務

子任務	分數	附加限制條件
1	5	根節點剛好有 M 個子節點。
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	無其他限制。

在子任務 3 和 4 中，你的分數取決於 C 的值，如下所示。

子任務 3 (25 分)

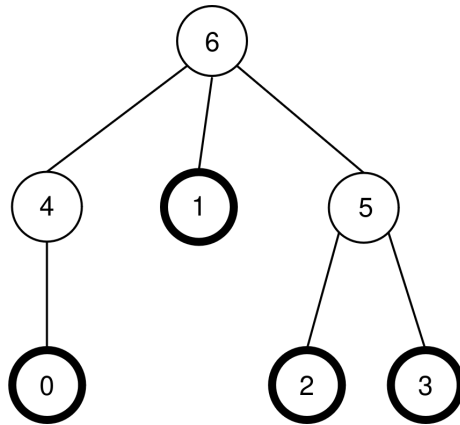
限制	分數
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

子任務 4 (60 分)

限制	分數
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

範例

考慮與題目描述中相同的機器，因此 $N = 7$ 、 $M = 4$ 以及 $P = [4, 6, 5, 5, 6, 6]$ 。該機器再次如以下圖所示：



評分器會發出以下呼叫：

```
find_structure(4)
```

該程序可以發出以下一連串的呼叫：

1. **insert(0, 0)**：葉節點0處於空間狀態，因此將一個值為0的球放置於葉節點0。節點 $P[0] = 4$ 處於空間狀態，因此球移動至節點4。節點 $P[4] = 6$ 處於空間狀態，因此球移動至節點6。由於節點6是根節點，因此移動停止。此呼叫返回true。
2. **insert(3, 20)**：葉節點3為空，因此將值為20的球放置於葉節點3。節點 $P[3] = 5$ 為空，因此球移動至節點5。由於 $P[5] = 6$ 已被佔用，移動停止。此呼叫返回true。
3. **insert(1, 10)**：葉節點1為空，因此將值為10的球放置於葉節點1。由於 $P[1] = 6$ 已被佔用，因此球仍留在節點1。此呼叫返回true。
4. **insert(2, 30)**：葉節點2為空，因此將值為30的球放置於葉節點2。由於 $P[2] = 5$ 已被佔用，因此球仍留在節點2。此呼叫返回true。
5. **insert(1, 25)**：葉節點1已被佔用，因此球不會被放置於葉節點1。此呼叫返回false。
6. **insert(0, 20)**：葉節點0為空，因此將值為20的球放置於葉節點0。節點 $P[0] = 4$ 為空，因此球移動至節點4。由於 $P[4] = 6$ 已被佔用，移動因此停止。此呼叫返回true。

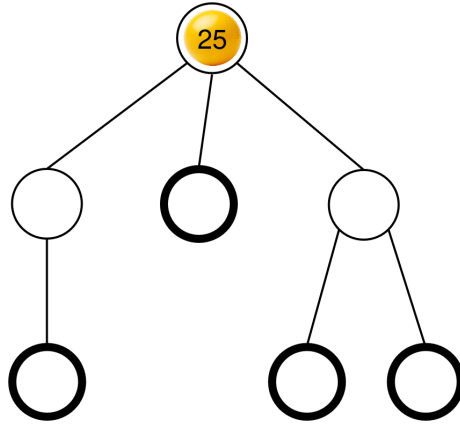
最終的球配置已在題目說明中顯示。

接下來，此程序可呼叫：

```
collect()
```

此呼叫的執行過程已在任務說明中闡述，因此此呼叫會傳回 $S = [0, 10, 20, 30, 20]$ 。之後，機器再次處於空狀態。

接著，程序可呼叫 **insert(2, 25)**。葉節點2為空，因此將值為25的球放置於葉節點0。該球隨後移動至節點5，接著移動至節點6，並在此停止。此呼叫會傳回true。球體的最終配置如以下圖所示。



最後，此程序可呼叫 `collect()`，該函式會傳回 $S = [25]$ 並清空機器。

`find_structure` 可能返回兩種正確的父陣列： $R = P = [4, 6, 5, 5, 6, 6]$ （對應標籤 $L = [0, 1, 2, 3, 4, 5, 6]$ ）以及 $R = [5, 6, 4, 4, 6, 6]$ （對應標籤 $L = [0, 1, 2, 3, 5, 4, 6]$ ）。在此範例中， $K = 2$ 與 $B = 30$ ，因此 $C = 32$ 。

樣例評分程式

輸入格式：

```
N M
P[0] P[1] P[2] ... P[N-2]
```

輸出格式：

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

此處， Q 即為 `find_structure` 所返回的陣列 R 的長度。